

AI活用を前提とした開発ライフサイクルへの移行

システム部 提案資料

本ディスカッションの背景

- きっかけ： Web機能追加のロードマップのディスカッションのなかで、役員の大樹さんから「世間ではAIによる開発の実績が出始めているのに、なぜバリューブックスでは実績が出せていないのか？」と質問された
- 本資料の目的： 「なぜバリューブックスでAI活用による開発で成果が現れていないか？」に対して、
 - 原因を現状の課題として列挙し、
 - 何を目指すべきか（What） とどうすべきか（How） を説明する

Goal

新機能をスピーディーかつ高品質にデリバリし、ビジネスへの貢献度を最大化する。

これを実現する手段として、AI活用を前提とした開発ライフサイクルへの移行を進める。

なぜAI活用による開発で成果が出せていないか

一言でいえば： AI活用スキルの不足に加え、組織として再現性のある仕組みができていない。さらにAIの効果が発揮しにくいシステム構造になっている。

ビジネスへの影響： デリバリが遅く、ビジネス側が独自に対応 → セキュリティリスクや不要な人的コストの原因に。

→ この問いへの答えが、次の 3つの構造的課題 である。

#	課題	キーワード
1	属人化・エンジニアリングの未成熟	人
2	AI活用が個人レベルにとどまり、プロセス全体が向上しない	プロセス
3	技術的負債により調査・検証に時間の大半を消費	システム

原因1：属人化・エンジニアリングの未成熟

→ AIを組織に組み込む「余裕」も「土台」もなく、AIファースト開発が進まない。

現状

- Web開発を自走できるのは実質2名。2パイプラインが上限
- リソース配分が不均一（バックヤードは逼迫していない）
- 「できる人がやった方が早い」が常態化 → スキル格差の固定化
- 負荷・稼働が可視化されておらず、経営判断の材料がない

→ なぜAIファースト開発が進まないのか

- 余裕がない：2名に集中 → AI活用の整備に割く時間がない
- 土台がない：他メンバーがAIを使うためのコンテキストが未整備
- → パイプライン数は2のまま、組織としてAIファースト開発が進んでいない

原因2：AI活用が個人レベルの業務効率化にとどまる

現状

- AI活用は個人レベルにとどまり、開発プロセス全体のスピードは向上していない
- 再現性のあるプロセスが未確立。ナレッジは暗黙知のまま
- ナレッジがチームに共有される文化もない

→ なぜ個人レベルにとどまるのか

- 業務知識・設計意図が個人の頭の中にしかない → AIに渡せる共有コンテキストがなく、各自の知識範囲でしかAIを活用できない
- AIの使い方も個人の試行錯誤に依存 → AI活用の領域でも属人化が再生産される

原因3：技術的負債

現状

- 古本販売主体のシステムに間に合わせて追加 → 密結合で影響範囲が不明な一枚岩
- テストが存在しない → 影響範囲を人手で調査、デリバリの大半を調査・検証に消費
- 品質の担保も個人の注意力に依存
- 技術的負債は一般的な課題（※事例・一般論は【参考】参照）

→ なぜAIで解決できないのか

- AIはコード生成が得意だが、実装は工数全体の一部にすぎない
- テストがなければAI生成コードの正しさも検証できず、人手の確認に戻る
- 調査・検証のボトルネックが残る限り、劇的な改善は不可能

3つの課題に対する対応策（What）

原因1（エンジニアリング未成熟）と原因2（AI活用が個人レベル）は関連が深い。

制約を取り払い、ビジネス目標をゼロベースでヒアリングし要件を再定義。AIファーストなアプローチで設計・実装を進める。

補足：既存システムでAIが機能しないのは前提条件（テスト・ドキュメント・明確な境界）の欠如が原因。新システムではこれらを最初から組み込む。

課題	対応策
属人化・エンジニアリング未成熟	人がボトルネックにならない仕組みを設計
AI活用が個人レベル	AIファースト開発ベースのプロセス確立
技術的負債	アーキテクチャ見直し（DDD等）→ 変更容易性向上
AIスキルの個人差	組織全体のAIスキル向上

これまでの取り組み

1. 業務構造に基づいたシステム設計手法の導入

チーム内で25回の勉強会・ワークショップを実施。

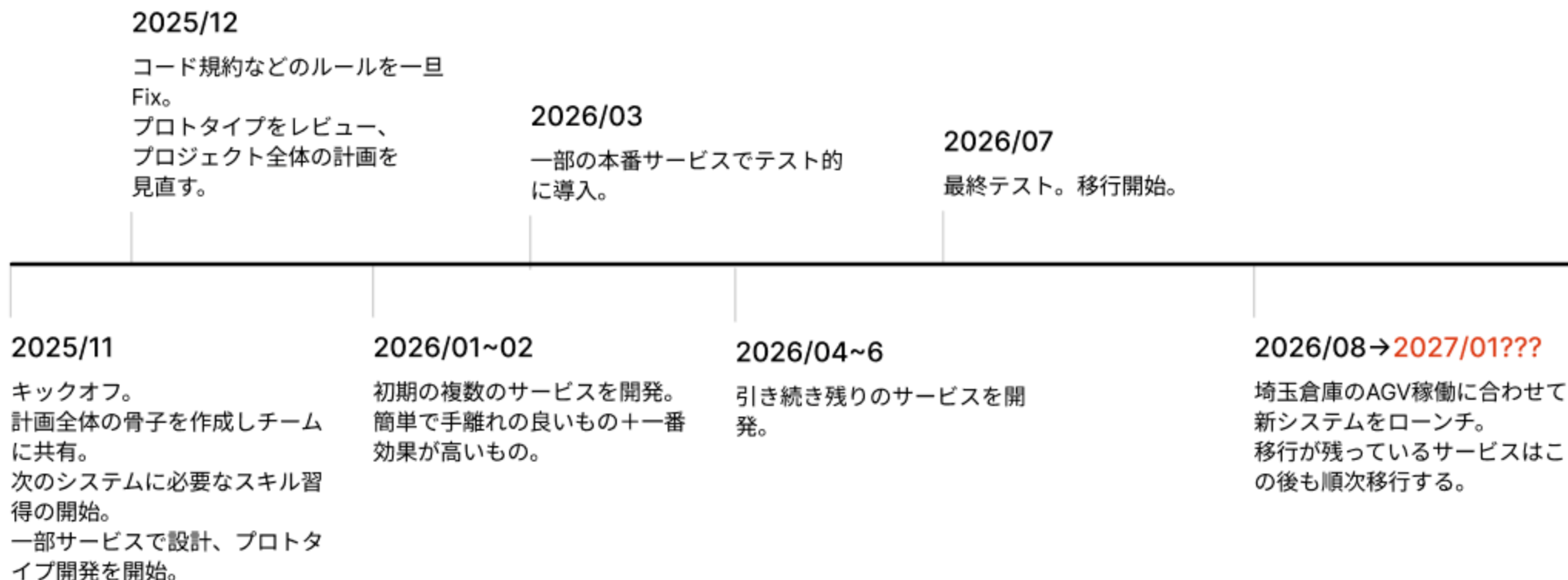
- イベントストーリーミング：業務プロセスの可視化とソフトウェアアーキテクチャ決定のためのワークショップ
- DDD勉強会：チーム全員が業務構造とシステム設計をつなぐ共通の考え方・言語を獲得
- 各個人のAI活用共有会：メンバー間でAI活用のナレッジを共有
- 業務境界の整理が進み、新システムへの移行単位の土台が整った

2. 要件定義・システム設計

現行システムの機能追加と技術的負債解消を両立するための設計が固まり、一部で着手を開始。

マイルストーン (TBD)

マイルストーン (TBD)



既存プランの見直しが必要な理由

活動の方向性は正しいが、2つの観点で見直しが必要。

1. 実績が出ていない

- ビジネスへの具体的な効果がまだ出ていない
- → まずは実績を出すことを優先し、成果で信頼を獲得する

2. 本質的なニーズを捉えられていない

- 現在の要件はシステム制約を前提に作られており、本当にやりたいことが反映されていない可能性
- → 制約を取り払い、ゼロベースで要件を再定義する

今後の進め方：3案の比較

本質的な業務要件の再取得と、AIファースト開発を活かしやすい設計への転換をどう実現するか－3案を比較する。

本節でいう「リスク」の意味

移行によって業務にネガティブな影響を与えるリスクを指す。

- サービス停止・品質低下、リリース遅延、コスト超過、運用混乱、顧客・社内への影響など
- 技術的負債の増大や属人化の継続は、別観点（持続性・負債解消）として表で扱う

案1：移行しない

既存システムのままAIを適用する

- 各自のAI練度を高める活動は継続し、調査やコーディングには一定の効果が見込める
- 暗黙知をAIが利用できるコンテキスト（ドキュメント・ルール・テスト・設計メモ）として顕在化し、AIツールで開発・改修を進める

観点	評価
コスト・リスク	新規インフラ不要。既存を壊さない
技術的負債	解消されない。調査・検証のボトルネックも残る
AI活用のしやすさ	個人レベルの調査・コーディングには一定の効果。プロセス全体の改善は限定的
持続性	負債は増え続ける

案2：ゆるやかな移行

業務単位（バーチャルスライス）で順次移行する

- 業務単位ごとにスクラッチで作り替え、完成したものから順に切り替えていく
- まず既存システムとの影響が少なく、かつビジネス効果が高い新刊販売から着手

観点	評価
コスト・リスク	業務単位で切り替えるためリスクが小さい。影響範囲を限定できる
技術的負債	移行した業務から順に解消。スコープが可視化される
AI活用のしやすさ	新規コードはクリーンでAIファースト開発と相性が良い
DDDとの整合	業務境界の整理をそのまま移行の境界に使える

向いているケース： サービスを止めずに、リスクを抑えながら段階的に成果を出したい場合

案3：ビッグバン移行

既存とは別に全システムをスクラッチで構築し、業務を段階的に移行する

- 一般的には悪手とされるアプローチだが、スピードは最も速い
- 全業務を一から設計・実装するため、設計の自由度は最大
- クリーンなコードベースでAIファースト開発の恩恵を最も受けやすい
- 移行は段階的に進める（例：3割を先行移行 → 残りを順次）
- 移行期は新旧オペレーション並行の可能性あり。拠点分け等で緩和可能

観点	評価
コスト・リスク	リスク最大。並行運用・データ整合性・オペレーション二重化
技術的負債	新システムで全面解消
AI活用のしやすさ	最も高い。全体がクリーンなコードベース

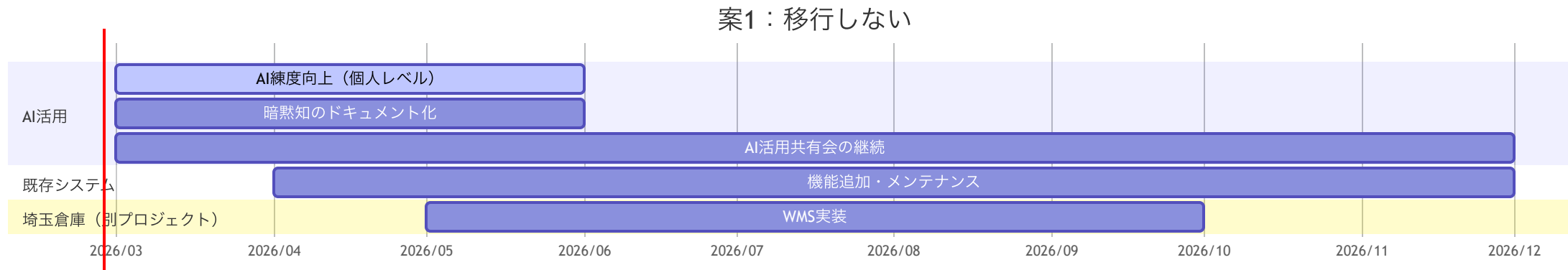
3案の位置づけ

案	リスク	スピード	技術的負債の解消	AI活用のしやすさ
1：移行しない	低	変わらない	不可	個人レベルで限定的
2：ゆるやかな移行	中	段階的に向上	移行した業務から順に解消	新規部分で高い
3：ビッグバン移行	高	最も速い	全面解消	最も高い

- 案1： 予算・時間が取れない場合の暫定策。AI練度向上は継続
- 案2： リスクと効果のバランスが良い。新刊販売から着手し段階的に成果を出す
- 案3： 最速だがリスク最大。現場オペレーション二重化の負荷も伴う

案1のマイルストーン：移行しない

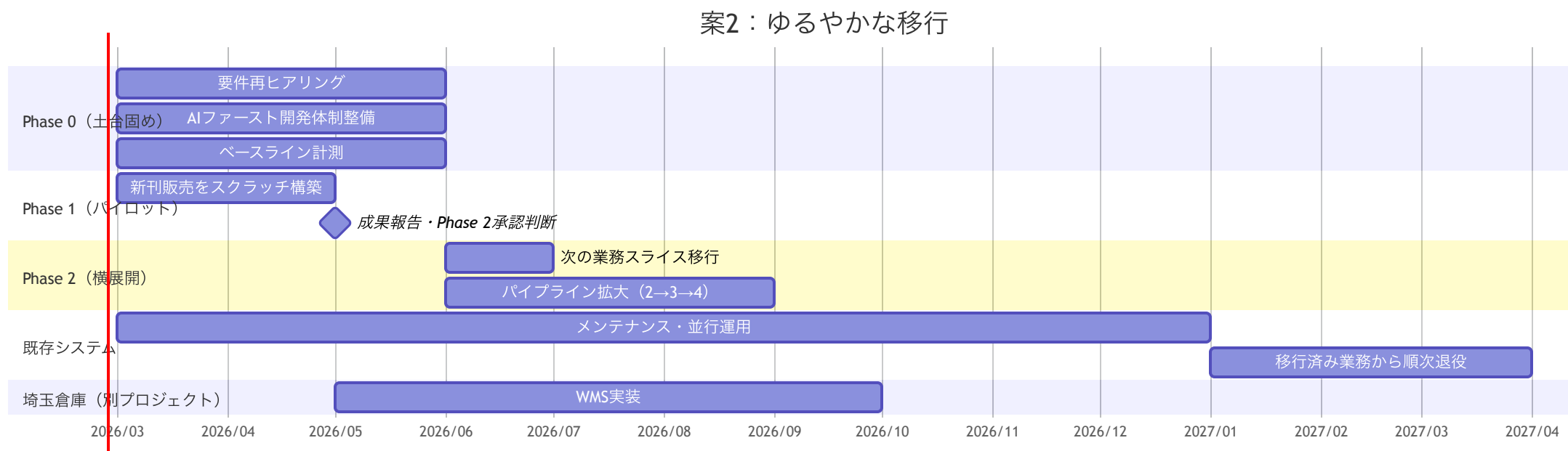
既存システムのまま、AI練度向上と暗黙知の顕在化に注力する。



- 技術的負債は解消されず、調査・検証のボトルネックは残る
- AI活用の効果は個人レベルの調査・コーディング効率化に限定
- 予算・時間が取れない場合の暫定策

案2のマイルストーン：ゆるやかな移行

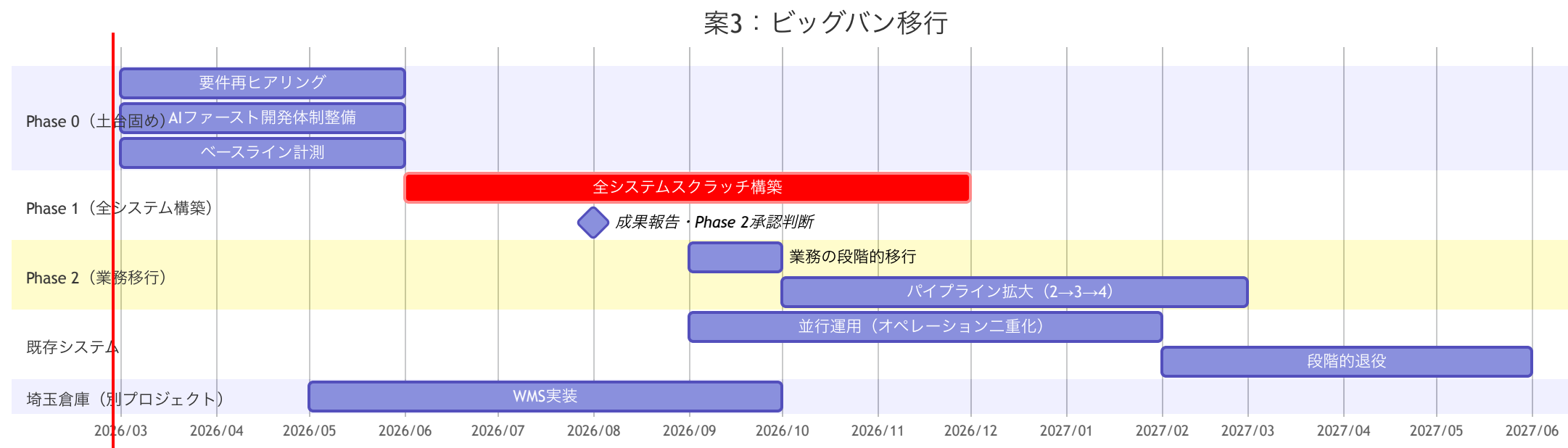
業務単位（バーチャルスライス）で順次移行。新刊販売から着手。



- リスクと効果のバランスが良い。影響範囲を業務単位で限定
- 新規コードはAIファースト開発と相性が良い

案3のマイルストーン：ビッグバン移行

既存とは別に全システムをスクラッチ構築し、業務を段階的に移行。



- 最速だがリスク最大。設計の自由度は最も高い
- 移行期は拠点分け等でオペレーション二重化を緩和

将来の組織構成：事業軸のチーム体制

現状の課題

「システム部」と「ビジネス部門」が分離 → 部門間の依頼・調整で開発

目指す姿

事業領域（販売、買取、マーケティング等）ごとに、
ビジネスメンバーとエンジニアが同じチームに所属する体制。

- ビジネスが「何を実現したいか」を起点、エンジニアが「どう実現するか」を担う
- チーム内で意思決定が完結 → 部門間調整コストの解消
- エンジニアがビジネスの成果に直接コミット
- トップラインの成長にシステム部が能動的に貢献

ローンチ後の展開

新システムローンチ後の取り組みを想定。

今年度

- EXOTEC（AGV）連携：倉庫・物流でのAGV導入
- 自動査定：買取査定の自動化
- 物流最適化：上田・埼玉の拠点効率化
- パーソナライゼーション：利用者に合わせた体験最適化
- Proの社外展開：書店向けサービスの外部提供

来年度以降（例）

- 社内業務アプリ・マーケティングクラウド業務の内製化
- 積読チャンネルでのライブコマース

投資・コスト

項目	月額（概算）	備考
AIツール	約11万円（\$760）	Claude Max \$200×3名、Pro \$20×8名
インフラダブルコスト	最大120万円	既存＋新システムの並行稼働
シニアエンジニア採用	60万円+	4～5月入社目標
JSL契約継続	250～300万円	既存契約の継続（増分ではない）

純粋な増分コスト

AIツール＋インフラ＋シニアエンジニア ÷ 最大約191万円/月

ディスカッション

1. それぞれの原因～対策方針をレビューし、訂正・補足する点はないか？
2. 案1～3の選択肢はどれが良さそうか？ または別の案があるか？
3. 今後の進め方についても方向性は問題ないか？

【参考】分析・データ基盤の改善見込み

分析・データ基盤の現状

本計画のスコープ外。システム再構築により改善が見込める領域。

現状の課題

- メインDBをLTV最大化のマーケティング・BigQuery（ユーザー獲得・定型レポ）で利用
- データをそのまま各部署に渡しており、分析に即したデータフォーマットになっていない

構造的な問題

- マスターテーブルとログだけのデータ構造がそのまま分析側に
- アプリケーション起因のデータモデルをビジネス側が解釈・加工することにコストがかかっている

分析基盤：本プロジェクトによる改善効果

根本原因

アプリのデータモデルがそのまま分析側に → 分析ユースケース起点のデータ設計がされていない

本プロジェクトで解消できること

- 分析ユースケースに基づくデータ設計の導入 → 解釈・加工コストを解消
- DDDの業務モデリングにより、分析に適したデータ構造（スタースキーマ等）も同時に設計
 - ドメインイベント → ファクト、エンティティ → ディメンション
- データが業務の意味を反映 → AIによる自然言語でのデータ問い合わせが容易に
- リアルタイム性向上 → ビジネスダッシュボードの提供が可能に

【参考】技術戦略

技術戦略：選定方針

- ベースとする技術スタックをあらかじめ決める
- 各ドメインの要件に応じて、そのドメインに限り別の技術を選定する権利を認める
- 選定時にはなぜその技術スタックを選んだかを **ADR (Architecture Decision Record)** として記録することが必須
- 後から理由を追えるようにし、AIファースト開発時のコンテキストとしても利用

技術戦略：推奨技術スタック

分類	推奨技術
Backend	TypeScript (Express)
Frontend	React、Electron (デスクトップアプリ)、Flutter (モバイルアプリ)
DB	PostgreSQL、DynamoDB、Valkey
アプリ基盤	Container
AWS サービス	ECS (Fargate)、ALB、RDS、DynamoDB、SNS、SQS、EventBridge、S3、Secrets Manager、Glue、DMS
開発環境	Docker + LocalStack

【参考】 技術的負債とAI活用に関する事例・一般論

技術的負債の多いモノリスとAIの限界

一般論

技術的負債の多いモノリスではAIファースト開発に限界があると国内外で報告されている。AIはコード断片の最適化に偏り、影響範囲や設計意図の理解が弱い。テスト不足だと検証もできず負債を増やしうる。設計・文脈を整えてからAIを組み合わせる必要がある。

事例

#	事例	要点
1	設計軽視のAI導入	1年後に保守不能なシステムに陥った報告
2	モノタロウの報告	レガシーにはAIが使えない・壊すリスクのジレンマ
3	海外の技術論考	大規模レガシー近代化はAI単体では失敗、設計と人的判断が不可欠